

Complete Workflow

Converting Files into Multilingual Audio

A Detailed Technical Guide for Educational Audio Systems

Project Type	Educational Interactive Map & Quiz Device	Languages	6 (EN, HI, MR, BN, GU, SA)
--------------	---	-----------	----------------------------

Table of Contents

- 1. Project Overview & Goals
- 2. Data Storage Architecture
- 3. Translation Process
- 4. Text-to-Speech Generation with gTTS
- 5. Audio Format Conversion with FFmpeg
- 6. File Naming Conventions
- 7. Multithreading for Performance
- 8. Sanskrit Audio — Special Handling
- 9. Output Categories
- 10. End-to-End Workflow Summary
- 11. Tools Reference Table
- 12. Best Practices & Tips

1. Project Overview & Goals



What Was Built?

This project delivers a fully automated pipeline that converts plain English text into audio files in **six languages**: English, Hindi, Marathi, Bengali, Gujarati, and Sanskrit. The audio powers an embedded educational device — a physical interactive map and quiz module used in schools.

Core Objectives

- Support multilingual learners with native-language audio prompts.
- Automate all translation and speech generation — no manual recording.
- Produce hardware-compatible WAV files for the embedded device.
- Process hundreds of audio files in parallel for speed.

✓ **Key Point:** The entire pipeline runs end-to-end in Python, from raw text to ready-to-flash WAV files, with no human intervention needed after the initial setup.

2. Data Storage Architecture

All content — questions, answers, instructions, state names, numbers, and quiz modes — is stored as **Python dictionaries**. This choice ensures fast key-based lookup, easy iteration, and straightforward serialization.

Why Python Dictionaries?

- **Speed:** $O(1)$ average-time key lookups, critical when generating hundreds of audio entries.
- **Flexibility:** Nested dicts allow storing all language variants under a single question key.
- **Readability:** Human-editable without a database setup.

Example Data Structure:

```
questions = {
  'q1': {
    'english': 'What is the capital of Maharashtra?',
    'hindi': 'महाराष्ट्र की राजधानी क्या है?',
    'marathi': 'महाराष्ट्र की राजधानी को काय म्हणता?',
    'bengali': 'মহারাষ্ট্রের রাজধানী কী?',
    'gujarati': 'મહારાષ્ટ્રની રાજધાની કયું?',
    'sanskrit': 'महाराष्ट्र की राजधानी कौनसा शहर है?'
  },
  ...
}
```

3. Translation Process

Translating educational content accurately requires more than machine translation alone. The project used a **hybrid approach**: AI-assisted bulk translation followed by manual review by native speakers.

Language	Script	Translation Method	Notes
Hindi	Devanagari	AI-assisted + review	Most widely available NLP support
Marathi	Devanagari	AI-assisted + manual	Shares script with Hindi; dialect care needed
Bengali	Bengali	AI-assisted + review	Second most spoken Indian language
Gujarati	Gujarati	AI-assisted + manual	Distinct script; fewer AI tools
Sanskrit	Devanagari	Manual by scholar	Classical language; gTTS uses Hindi TTS

■ **Note:** Machine translation tools (Google Translate, AI models) handle the bulk work, but educational content requires expert review — especially for regional languages where colloquialisms and grade-appropriate vocabulary matter.

4. Text-to-Speech Generation with gTTS

gTTS (Google Text-to-Speech) is an open-source Python library that interfaces with Google's TTS API to convert text into spoken MP3 audio. It was chosen for this project because it supports all the required Indian languages and is trivially simple to integrate into a Python automation script.

How gTTS Works

- **Input:** A string of Unicode text + a BCP-47 language code (e.g., *hi* for Hindi, *mr* for Marathi).
- **Network call:** gTTS sends the text to Google's servers and receives an MP3 audio stream.
- **Output:** The MP3 is saved locally with a structured filename.

Language Codes Used:

Language	gTTS Code	Example Text
Hindi	hi	आपका स्वागत है। आप कैसे हैं?
Marathi	mr	आपका स्वागत आहे। आप कसे आहात?
Bengali	bn	আপনার স্বাগতম। আপনি কেমন আছেন?
Gujarati	gu	આપકા સ્વાગત છે। આપ કેમ છો?
Sanskrit	hi*	* Uses Hindi voice (see Section 8)

Sample Python Code:

```
from gtts import gTTS
import os

def generate_audio(text, lang_code, output_path):
    tts = gTTS(text=text, lang=lang_code, slow=False)
    tts.save(output_path)
    print(f'Saved: {output_path}')

# Example: Hindi question
generate_audio('■■■■■■■■■■ ■■ ■■■■■■■■ ■■■■ ■■?', 'hi', 'q1_hindi.mp3')
```

✓ **Key Point:** gTTS generates MP3 files. For embedded hardware that requires WAV, a second conversion step using FFmpeg is needed (see Section 5).

5. Audio Format Conversion with FFmpeg

FFmpeg is a powerful open-source multimedia framework that can decode, encode, transcode, mux, demux, stream, filter, and play virtually any audio or video format. Here, it converts gTTS-generated **MP3 files** → **WAV files** for embedded hardware compatibility.

Why WAV for Embedded Hardware?

- **No decoding overhead:** WAV (PCM) is uncompressed and plays directly without a decoder chip.
- **Universal support:** Microcontrollers and SoCs (Arduino, ESP32, Raspberry Pi) all read WAV natively.
- **Deterministic timing:** Uncompressed audio has no buffering/latency variability — critical for quiz synchronization.

FFmpeg Conversion Command:

```
# Convert a single MP3 to WAV (16-bit PCM, 22050 Hz, mono)
ffmpeg -i input.mp3 -ar 22050 -ac 1 -acodec pcm_s16le output.wav

# Parameters explained:
# -ar 22050 → sample rate (22.05 kHz – good quality, small file)
# -ac 1 → mono channel (saves space on embedded storage)
# -acodec pcm_s16le → 16-bit little-endian PCM (standard WAV)
```

Python Automation with subprocess:

```
import subprocess

def convert_mp3_to_wav(mp3_path, wav_path):
    cmd = [
        'ffmpeg', '-i', mp3_path,
        '-ar', '22050', '-ac', '1',
        '-acodec', 'pcm_s16le',
        wav_path, '-y' # -y = overwrite without asking
    ]
    subprocess.run(cmd, check=True, capture_output=True)
```

6. File Naming Conventions

A consistent, descriptive naming convention is essential when managing hundreds of audio files across six languages and multiple content categories. The project uses a structured pattern combining **content type** + **identifier** + **language suffix**.

Naming Pattern:

```
{category}_{id}_{language}.wav
```

Examples:

```
question_01_hindi.wav
```

```
solution_01_marathi.wav
```

```
state_maharashtra_gujarati.wav
```

```
number_05_bengali.wav
```

```
mode_quiz_sanskrit.wav
```

```
instruction_start_hindi.wav
```

Suffix	Language	Script
_hindi.wav	Hindi	Devanagari
_marathi.wav	Marathi	Devanagari
_bengali.wav	Bengali	Bengali
_gujarati.wav	Gujarati	Gujarati
_sanskrit.wav	Sanskrit	Devanagari (via Hindi TTS)
_english.wav	English	Latin

■ **Note:** Avoid spaces in filenames — embedded firmware often cannot handle spaces in paths. Use underscores exclusively.

7. Multithreading for Performance

Processing hundreds of audio files sequentially would be slow — each gTTS call involves a network round-trip to Google's servers. **ThreadPoolExecutor** from Python's *concurrent.futures* module runs multiple TTS and FFmpeg jobs in parallel, dramatically cutting total processing time.

Threading Architecture:

- **15 worker threads** were used — enough to saturate typical network bandwidth without overwhelming the system.
- Each thread handles one audio file: translate → gTTS call → FFmpeg conversion → save.
- Thread-safe file writing: each thread writes to a unique file path, so no locking is needed.

Performance Comparison:

Approach	200 files (estimated)	Notes
Sequential	~60-90 minutes	One network call at a time
ThreadPoolExecutor	~5-8 minutes	15 parallel network calls
Speedup	~10-15x faster	Near-linear scaling with threads

Sample Code:

```
from concurrent.futures import ThreadPoolExecutor
from gtts import gTTS
import subprocess

def process_one(args):
    text, lang, mp3_path, wav_path = args
    gTTS(text=text, lang=lang).save(mp3_path)
    subprocess.run(['ffmpeg', '-i', mp3_path, '-ar', '22050',
                    '-ac', '1', '-acodec', 'pcm_s16le', wav_path, '-y'],
                  check=True, capture_output=True)

tasks = [...] # list of (text, lang, mp3, wav) tuples

with ThreadPoolExecutor(max_workers=15) as executor:
    executor.map(process_one, tasks)
```

8. Sanskrit Audio — Special Handling

Sanskrit presents a unique challenge: **gTTS does not support Sanskrit as a language code**. Sanskrit lacks a BCP-47 code recognized by Google's TTS API. However, Sanskrit is written in **Devanagari script** — the same script used for Hindi — and the two languages share a large portion of their phonological system.

The Workaround:

1 Write Sanskrit in Devanagari

Sanskrit text is composed and stored in Devanagari Unicode, identical in encoding to Hindi text.

2 Pass to gTTS with lang='hi'

The Hindi TTS model reads the Devanagari characters. Because Sanskrit phonology is a subset of Classical Sanskrit-derived Hindi phonemes, the pronunciation is largely correct.

3 Convert and Save as `_sanskrit.wav`

The resulting audio is converted via FFmpeg and saved with the `_sanskrit.wav` suffix, making it identifiable in the firmware.

■ **Note:** The Hindi TTS voice will render Sanskrit with a slight modern Hindi accent. For most educational contexts this is acceptable. For high-fidelity Sanskrit pronunciation, a custom trained TTS model or a human voice recording would be needed.

9. Output Categories

The pipeline generates six distinct categories of audio files. Each category serves a specific role in the interactive device's user experience.

1 Quiz Questions

The core educational content. Each question is stored per language so the device can play the question in the student's selected language.

2 Solutions / Answers

Correct-answer audio played after the student responds. Includes full explanatory sentences, not just the answer word.

3**Instructions**

Onboarding prompts like 'Press the state you think is correct' or 'Quiz mode is now active.'
Critical for first-time users.

4**State Information**

Audio descriptions of Indian states — area, capital, key facts — triggered when a state is tapped on the map.

5**Counting Numbers**

Numbers 0–99 (or higher) in each language for score announcements and countdowns.

6**Quiz Modes**

Mode-selection prompts: 'Easy Mode', 'Hard Mode', 'Practice Mode', etc., read aloud when the mode button is pressed.

10. End-to-End Workflow Summary

Step 1	English Source Text All content is authored in English in Python dictionaries. This is the single source of truth.
Step 2	Translation English text is translated into 5 Indian languages (Hindi, Marathi, Bengali, Gujarati, Sanskrit) via AI tools and manual review.
Step 3	gTTS → MP3 Generation Python + gTTS sends each translated string to Google's TTS API and receives back an MP3 file per string per language.
Step 4	FFmpeg → WAV Conversion Each MP3 is converted to PCM WAV (16-bit, 22050 Hz, mono) via FFmpeg. This is done in the same worker thread immediately after MP3 generation.
Step 5	File Organization WAV files are moved to structured output folders organized by category and language.
Step 6	Flash to Device Final WAV files are copied to the embedded device's SD card or flash memory and addressed by filename in firmware.

English Text → Translation → gTTS MP3 → FFmpeg WAV → Organized Multilingual Audio Files

11. Tools Reference Table

Tool	Version / Type	Role in Workflow	Why Chosen
Python	3.8+	Master orchestrator; all glue code	Rich ecosystem, easy concurrency
gTTS	2.x (pip)	Text → MP3 speech synthesis	Free, multi-language Indian script support
FFmpeg	4.x+ (CLI)	MP3 → WAV format conversion	Industry standard, all codecs, scriptable
ThreadPoolExecutor stdlib		Parallel processing of 100s of files	No dependencies, simple API, 15 workers
Google Translate	Web / API	Bulk English → Indian lang translation	Broadest language coverage
AI Assistant	GPT / Claude	Translation review, Sanskrit help	Quality check, classical language aid

12. Best Practices & Tips



Rate Limiting

gTTS calls hit Google's servers. Add a small random delay (0.1–0.5 s) between requests in each thread to avoid IP throttling.



Error Handling

Wrap each gTTS and FFmpeg call in try/except. Log failures to a CSV so you can re-run only failed files without re-processing everything.



Dry Run Mode

Before generating all 600+ files, test with 5–10 entries across all languages to catch encoding and TTS issues early.



Storage Budget

A 3-second WAV at 22050 Hz mono $\approx$ 130 KB. 600 files $\approx$ 78 MB. Verify your embedded device has sufficient flash/SD capacity.



Unicode Handling

Always open Python source files with UTF-8 encoding (`# -*- coding: utf-8 -*-`) when hardcoding Indic script strings.



Backup MP3s

Keep the intermediate MP3 files. If you need a different WAV sample rate later, you can re-convert without repeating gTTS API calls.



Testing on Device

Play a sample from each language on the actual hardware before flashing the full set — some embedded DACs behave differently than desktop playback.

This document elaborates the complete multilingual audio pipeline for the educational interactive map and quiz device. The workflow — from English text to hardware-ready WAV files — is fully automated, scalable, and reproducible.